

Prüfungssimulation – Beispiel 2

Diese Probeprüfung orientiert sich am offiziellen Themenkatalog der IHK (gültig ab Oktober 2024) und an typischen Aufgaben der vergangenen Jahre. Die Aufgaben sind in deutscher Sprache formuliert und werden unmittelbar darunter auf Persisch erläutert. Zu jeder Teilaufgabe folgt ein Musterlösungs-Pseudocode.

Aufgabe 1 – Generisches Sortieren mit Comparator

Teil a – Generische Sortierfunktion

Aufgabe: Schreiben Sie einen Pseudocode für eine generische Sortierfunktion `sortiereListe`, die eine Liste `liste` mit Elementen vom Typ `T` und eine Vergleichsfunktion `comp` erhält. Verwenden Sie das Bubble-Sort-Verfahren und rufen Sie in jeder Vergleichsoperation die Funktion `comp` auf, um zu entscheiden, ob zwei Elemente vertauscht werden müssen. Die Funktion soll die Liste in-place aufsteigend sortieren.

Lösung

```
funktion sortiereListe(liste: List<T>, comp: Funktion(T, T): Integer): void
    n = liste.size()
    für i = 0 bis n - 2
        für j = 0 bis n - 2 - i
            // vergleiche benachbarte Elemente mit comp
            wenn comp(liste[j], liste[j + 1]) > 0 dann
                // Elemente vertauschen
                temp = liste[j]
                liste[j] = liste[j + 1]
                liste[j + 1] = temp
            ende wenn
        ende für
    ende für
endfunktion
```

توضیح سوال: این بخش از شما می‌خواهد یک تابع عمومی برای مرتب‌سازی بنویسید که لیستی از هر نوع داده‌ای را با استفاده از الگوریتم حبابی و تابع مقایسه‌گر مرتب کند.

توضیح پاسخ: در این کد ابتدا اندازه لیست تعیین می‌شود. حلقه بیرونی مشخص می‌کند چند بار لیست پیمایش شود و حلقه داخلی هر بار دو عنصر مجاور را با استفاده از تابع `comp` مقایسه می‌کند. اگر تابع مقایسه مقدار مثبت برگرداند، یعنی عنصر اول باید بعد از عنصر دوم قرار گیرد و عناصر جابجا می‌شوند. این روند ادامه می‌یابد تا لیست مرتب شود.

Teil b – Comparator für Mitarbeiter

Aufgabe: Schreiben Sie einen Pseudocode für eine Vergleichsfunktion

vergleicheMitarbeiter, die zwei Objekte vom Typ Mitarbeiter anhand ihres Gehalts vergleicht. Die Funktion soll einen positiven Wert zurückgeben, wenn das Gehalt von m_1 größer ist als das von m_2 , einen negativen Wert, wenn es kleiner ist, und 0, wenn beide gleich sind.

Lösung

```
funktion vergleicheMitarbeiter(m1: Mitarbeiter, m2: Mitarbeiter): Integer
    gehalt1 = m1.getGehalt()
    gehalt2 = m2.getGehalt()
    wenn gehalt1 > gehalt2 dann
        Rückgabe 1
    sonst wenn gehalt1 < gehalt2 dann
        Rückgabe -1
    sonst
        Rückgabe 0
    ende wenn
endfunktion
```

توضیح سؤال: این زیرمجموعه از شما می‌خواهد تابعی برای مقایسه دو شیء از کلاس Mitarbeiter بنویسید که بر اساس مقدار حقوق آن‌ها نتیجه مقایسه را برگرداند.

توضیح پاسخ: ابتدا حقوق هر کارمند در متغیرهای جداگانه ذخیره می‌شود. سپس با مقایسه این دو مقدار مشخص می‌شود که آیا حقوق m_1 بیشتر است، کمتر است یا برابر. به ترتیب مقادیر ۱، -۱ یا ۰ بازگردانده می‌شود تا ترتیب بین اشیاء مشخص شود.

Teil c – Vorteile von Generics

Aufgabe: Erläutern Sie kurz, warum die Verwendung von Generics und Vergleichsfunktionen (Comparator) die Flexibilität eines Sortieralgorithmus erhöht.

Lösung

Durch Generics kann dieselbe Sortierfunktion für unterschiedliche Datentypen wiederverwendet werden, ohne dass der Code dupliziert werden muss. Die Vergleichslogik wird in einer separaten Funktion gekapselt (Comparator), sodass der Sortieralgorithmus selbst unverändert bleibt und lediglich eine andere Vergleichsfunktion übergeben werden muss, um z. B. nach Gehalt, Name oder Datum zu sortieren.

توضیح سؤال: باید توضیح دهید چرا استفاده از نوع‌های کلی (Generics) و تابع مقایسه‌گر باعث انعطاف‌پذیری بیشتر در مرتب‌سازی می‌شود.

توضیح پاسخ: با استفاده از Generics همان کد مرتب‌سازی برای انواع مختلف داده قابل استفاده است و نیازی به نوشتن نسخه‌های جداگانه نیست. تابع مقایسه‌گر منطق مقایسه را از الگوریتم جدا می‌کند، بنابراین می‌توان با تغییر این تابع، معیار

Aufgabe 2 – Jahresverbrauchsstatistik

Teil a – Minimum, Maximum und Grenzwertüberschreiter

Aufgabe: Gegeben ist eine zweidimensionale Matrix `verbrauch`, bei der jede Zeile einem Benutzer und jede Spalte einem Monat entspricht. Schreiben Sie einen Pseudocode, der den minimalen Verbrauch `minVerbrauch`, den maximalen Verbrauch `maxVerbrauch` im gesamten Jahr und eine Liste `limitVerbraucher` mit den Indizes der Benutzer bestimmt, deren Jahresgesamtverbrauch über dem Grenzwert `limit` liegt. Die Funktion soll ein Objekt Jahresstatistik mit diesen Werten zurückgeben.

Lösung

```
funktion statistik(verbrauch: Array von Array von Integer, limit: Integer): Jahresstatistik
    minVerbrauch = +∞
    maxVerbrauch = -∞
    limitVerbraucher = leere Liste
    für i = 0 bis verbrauch.length - 1
        summe = 0
        für j = 0 bis verbrauch[i].length - 1
            wert = verbrauch[i][j]
            summe = summe + wert
            wenn wert < minVerbrauch dann
                minVerbrauch = wert
            ende wenn
            wenn wert > maxVerbrauch dann
                maxVerbrauch = wert
            ende wenn
        ende für
        wenn summe > limit dann
            füge i zu limitVerbraucher hinzu
        ende wenn
    ende für
    Rückgabe neues Jahresstatistik(minVerbrauch, maxVerbrauch, limitVerbraucher)
endfunktion
```

توضیح سؤال: این قسمت از شما می‌خواهد از میان جدول مصرف کاربران حداقل، حداکثر و کاربران پرمصرف (بیشتر از حد مشخص) را پیدا کنید.

توضیح پاسخ: ابتدا متغیرهای `minVerbrauch` و `maxVerbrauch` مقداردهی اولیه می‌شوند. سپس هر سطر (هر کاربر) و هر ستون (ماه) پیمایش می‌شود. مجموع مصرف کاربر در متغیر `summe` جمع می‌شود و در عین حال کمترین و بیشترین مقدار

تکماه بررسی می‌شود. اگر مجموع مصرف سالانه کاربر بیشتر از حد limit باشد، اندیس کاربر در لیست limitVerbraucher ذخیره می‌شود. در پایان یک شیء Jahresstatistik حاوی این داده‌ها بازگردانده می‌شود.

Teil b – Durchschnittsverbrauch

Aufgabe: Schreiben Sie einen Pseudocode, der für jede Zeile der Matrix verbrauch den Durchschnittsverbrauch des jeweiligen Benutzers berechnet und außerdem den durchschnittlichen Verbrauch aller Benutzer insgesamt ermittelt. Die Funktion soll zwei Werte zurückgeben: ein Array durchschnittProUser und einen Gesamtwert gesamtDurchschnitt.

Lösung

```
funktion berechneDurchschnitt(verbrauch: Array von Array von Integer): (Array von Double,  
    anzahlBenutzer = verbrauch.length  
    anzahlMonate = verbrauch[0].length  
    durchschnittProUser = neues Array von Double der Länge anzahlBenutzer  
    summeAlle = 0  
    gesamtWerte = 0  
    für i = 0 bis anzahlBenutzer - 1  
        summe = 0  
        für j = 0 bis anzahlMonate - 1  
            summe = summe + verbrauch[i][j]  
        ende für  
        durchschnittProUser[i] = summe / anzahlMonate  
        summeAlle = summeAlle + summe  
        gesamtWerte = gesamtWerte + anzahlMonate  
    ende für  
    gesamtDurchschnitt = summeAlle / gesamtWerte  
    Rückgabe (durchschnittProUser, gesamtDurchschnitt)  
endfunktion
```

توضیح سؤال: باید میانگین مصرف ماهیانه برای هر کاربر و میانگین کلی برای همه کاربران محاسبه شود.

توضیح پاسخ: در ابتدا اندازه جدول و تعداد کاربران تعیین می‌شود. برای هر کاربر مجموع ماهانه محاسبه و سپس بر تعداد ماه‌ها تقسیم می‌شود تا میانگین وی به‌دست آید. در طول این فرآیند مجموع همه مقادیر و تعداد کل مقادیر برای محاسبه میانگین کلی جمع‌آوری می‌شود. در پایان، آرایه میانگین‌ها و مقدار میانگین کلی بازگردانده می‌شود.

Teil c – Benutzer mit maximalem Gesamtverbrauch

Aufgabe: Schreiben Sie einen Pseudocode, der aus der Matrix verbrauch den Index des Benutzers ermittelt, dessen Gesamtverbrauch über alle Monate am höchsten ist. Bei mehreren Benutzern mit gleichem Spitzenwert kann der erste Index zurückgegeben werden.

Lösung

```

funktion findeTopVerbraucher(verbrauch: Array von Array von Integer): Integer
    besterIndex = -1
    besterWert = -∞
    für i = 0 bis verbrauch.length - 1
        summe = 0
        für j = 0 bis verbrauch[i].length - 1
            summe = summe + verbrauch[i][j]
        ende für
        wenn summe > besterWert dann
            besterWert = summe
            besterIndex = i
        ende wenn
    ende für
    Rückgabe besterIndex
endfunktion

```

توضیح سؤال: در این بخش باید کاربری با بیشترین مصرف سالانه را بیابید.

توضیح پاسخ: برای هر کاربر مجموع مصرف محاسبه می‌شود و اگر این مقدار از بهترین مقدار موجود بیشتر باشد، مقدار جدید به عنوان بهترین ثبت می‌شود و اندیس کاربر ذخیره می‌شود. در پایان اندیس کاربر با بیشترین مصرف برگردانده می‌شود.

Aufgabe 3 – Prüfziffer und Zahlensysteme

Teil a – Prüfziffer berechnen

Aufgabe: Schreiben Sie einen Pseudocode, der aus einem Array *ziffern* von ganzen Ziffern (0–9) eine Prüfziffer nach dem Verfahren berechnet: Jede Ziffer wird von links nach rechts mit den Gewichten 1, 2, 3, ... multipliziert. Die Prüfziffer ist der Rest der Summe der gewichteten Ziffern modulo 10.

Lösung

```

funktion berechnePruefziffer(ziffern: Array von Integer): Integer
    summe = 0
    für i = 0 bis ziffern.length - 1
        gewicht = i + 1
        summe = summe + ziffern[i] × gewicht
    ende für
    pruefziffer = summe mod 10
    Rückgabe pruefziffer
endfunktion

```

توضیح سؤال: این قسمت از شما می‌خواهد که عدد کنترلی را با ضرب رقم‌های یک شماره در وزن‌های متوالی و سپس گرفتن باقیمانده تقسیم بر ۱۰ محاسبه کنید.

توضیح پاسخ: یک حلقه رقم‌های آرایه را از چپ به راست پیمایش می‌کند. وزن هر رقم یک عدد افزایشی از ۱ تا طول آرایه است. حاصل ضرب هر رقم و وزن به مجموع اضافه می‌شود. سپس مجموع بر ۱۰ تقسیم می‌شود و باقیمانده به عنوان رقم کنترلی بازگردانده می‌شود.

Teil b – Prüfziffer prüfen

Aufgabe: Gegeben seien ein Array `ziffern` und eine berechnete Prüfziffer `p`. Schreiben Sie einen Pseudocode, der prüft, ob `p` die korrekte Prüfziffer für `ziffern` ist. Die Funktion soll `true` zurückgeben, wenn die Prüfziffer stimmt, andernfalls `false`.

Lösung

```
funktion pruefePruefziffer(ziffern: Array von Integer, p: Integer): Boolean
    berechnet = berechnePruefziffer(ziffern)
    wenn berechnet = p dann
        Rückgabe wahr
    sonst
        Rückgabe falsch
    ende wenn
endfunktion
```

توضیح سؤال: لازم است بررسی کنید که آیا رقم کنترلی داده شده با مقدار محاسبه شده مطابقت دارد یا نه.

توضیح پاسخ: با استفاده از تابع محاسبه رقم کنترلی، مقدار صحیح رقم کنترلی محاسبه و با رقم ارائه‌شده مقایسه می‌شود. اگر دو مقدار برابر باشند، `true` و در غیر این صورت `false` برگردانده می‌شود.

Teil c – Umrechnung in binäre Darstellung

Aufgabe: Schreiben Sie einen Pseudocode, der eine ganzzahlige Dezimalzahl `n` in ihre Binärdarstellung umwandelt. Verwenden Sie dabei die Methode der wiederholten Division durch 2 und sammeln Sie die Reste.

Lösung

```
funktion dezimalZuBinaer(n: Integer): String
    wenn n = 0 dann
        Rückgabe "0"
    binaer = ""
    zahl = n
    während zahl > 0
        rest = zahl mod 2
        binaer = toString(rest) + binaer
    ende während
endfunktion
```

```
    zahl = zahl div 2
ende während
Rückgabe binaer
endfunktion
```

توضیح سؤال: باید عدد دهدهی را به صورت دودویی تبدیل کنید.

توضیح پاسخ: اگر مقدار صفر باشد مستقیماً "0" برگردانده می‌شود. در غیر این صورت، با استفاده از تقسیم متوالی بر ۲ و افزودن باقیمانده‌ها به ابتدای رشته، رشته دودویی تولید می‌شود. حلقه ادامه می‌یابد تا مقدار zahl صفر شود.

Aufgabe 4 – Suchalgorithmen

Teil a – Lineare Suche

Aufgabe: Schreiben Sie einen Pseudocode für die lineare Suche in einem Array A von Ganzzahlen. Die Funktion soll den Index des gesuchten Wertes x zurückgeben oder -1, wenn der Wert nicht gefunden wird.

Lösung

```
funktion lineareSuche(A: Array von Integer, x: Integer): Integer
    für i = 0 bis A.length - 1
        wenn A[i] = x dann
            Rückgabe i
        ende wenn
    ende für
    Rückgabe -1
endfunktion
```

توضیح سؤال: باید جستجوی خطی را پیاده‌سازی کنید تا عنصر مورد نظر را در آرایه پیدا کنید.

توضیح پاسخ: این الگوریتم آرایه را از ابتدا تا انتها پیمایش می‌کند و اگر عنصر مورد نظر پیدا شود، اندیس آن بازگردانده می‌شود. اگر عنصر یافت نشود، -1 بازگردانده می‌شود.

Teil b – Binäre Suche

Aufgabe: Schreiben Sie einen Pseudocode für die binäre Suche in einem aufsteigend sortierten Array A von Ganzzahlen. Die Funktion soll den Index des gesuchten Wertes x zurückgeben oder -1, wenn der Wert nicht gefunden wird.

Lösung

```

funktion binaereSuche(A: Array von Integer, x: Integer): Integer
    links = 0
    rechts = A.length - 1
    während links ≤ rechts
        mitte = (links + rechts) div 2
        wenn A[mitte] = x dann
            Rückgabe mitte
        sonst wenn A[mitte] > x dann
            rechts = mitte - 1
        sonst
            links = mitte + 1
        ende wenn
    ende während
    Rückgabe -1
endfunktion

```

توضیح سؤال: این زیرمجموعه از شما می‌خواهد جستجوی دودویی را روی آرایه مرتب پیاده‌سازی کنید.

توضیح پاسخ: ابتدا محدوده جستجو با دو متغیر `links` و `rechts` مشخص می‌شود. در هر تکرار عنصر میانی با مقدار مورد نظر مقایسه می‌شود و بر اساس نتیجه محدوده جستجو نصف می‌شود. در صورت یافتن مقدار، اندیس آن بازگردانده می‌شود و در غیر این صورت، -1 بازگردانده می‌شود.

Teil c – Vergleich der Suchalgorithmen

Aufgabe: Vergleichen Sie lineare und binäre Suche hinsichtlich ihrer Zeitkomplexität und Anwendungsfälle.

Lösung

Die lineare Suche benötigt im schlimmsten Fall n Vergleiche ($O(n)$) und eignet sich für unsortierte Daten oder kleine Datenmengen, da sie einfach zu implementieren ist. Die binäre Suche ist deutlich schneller mit einer Laufzeit von $O(\log n)$, setzt jedoch voraus, dass die Daten bereits sortiert sind. Sie eignet sich für große Datenmengen, bei denen der Aufwand zur Sortierung gerechtfertigt ist.

توضیح سؤال: لازم است مزایا و معایب دو الگوریتم جستجو و زمان اجرای آن‌ها ذکر شود.

توضیح پاسخ: جستجوی خطی ساده است و برای آرایه‌های نامرتب یا کوچک مناسب است ولی در بدترین حالت باید همه عناصر را بررسی کند (پیچیدگی زمانی $O(n)$). جستجوی دودویی بسیار سریع‌تر است ($O(\log n)$) اما برای استفاده از آن باید آرایه مرتب باشد.

Aufgabe 5 – Rekursion vs Iteration

Teil a – Fakultät iterativ berechnen

Aufgabe: Schreiben Sie einen Pseudocode, der die Fakultät einer natürlichen Zahl n iterativ berechnet (also ohne Rekursion). Die Fakultät ist definiert als $n! = 1 \times 2 \times \dots \times n$.

Lösung

```
funktion fakultaetIterativ(n: Integer): Integer
    wenn n <= 1 dann
        Rückgabe 1
    ergebnis = 1
    für i = 2 bis n
        ergebnis = ergebnis × i
    ende für
    Rückgabe ergebnis
endfunktion
```

توضیح سؤال: باید تابعی بنویسید که مقدار فاکتوریل یک عدد را بدون استفاده از بازگشت محاسبه کند.

توضیح پاسخ: ابتدا برای n برابر یا کمتر از ۱ مقدار ۱ برگردانده می‌شود. سپس با استفاده از یک حلقه از ۲ تا n ، مقدار ergebnis به تدریج در i ضرب می‌شود تا فاکتوریل به دست آید.

Teil b – Fakultät rekursiv berechnen

Aufgabe: Schreiben Sie einen Pseudocode, der die Fakultät einer natürlichen Zahl n rekursiv berechnet.

Lösung

```
funktion fakultaetRekursiv(n: Integer): Integer
    wenn n <= 1 dann
        Rückgabe 1
    sonst
        Rückgabe n × fakultaetRekursiv(n - 1)
    ende wenn
endfunktion
```

توضیح سؤال: باید همان تابع فاکتوریل را این بار به صورت بازگشتی پیاده‌سازی کنید.

توضیح پاسخ: تابع بازگشتی ابتدا بررسی می‌کند که آیا مقدار n کوچکتر یا مساوی ۱ است؛ در این حالت ۱ برگردانده می‌شود که شرط توقف است. در غیر این صورت، n در نتیجه فراخوانی تابع با $n-1$ ضرب می‌شود تا فاکتوریل محاسبه شود.

Teil c – Vergleich Rekursion vs Iteration

Aufgabe: Erläutern Sie zwei Vorteile und zwei Nachteile der rekursiven gegenüber der iterativen Umsetzung am Beispiel der Fakultät.

Lösung

Rekursive Funktionen orientieren sich häufig näher an der mathematischen Definition und sind daher intuitiver und kürzer. Sie eignen sich gut für Probleme, die eine natürliche Teilstruktur besitzen (z. B. Bäume). Nachteile sind der höhere Speicherbedarf durch den Aufrufstapel und das Risiko eines Stackoverflow bei großen Werten; zudem kann die Performance im Vergleich zur iterativen Lösung schlechter sein.

توضیح سؤال: باید مزایا و معایب روش بازگشتی نسبت به روش تکراری را توضیح دهید.

توضیح پاسخ: استفاده از بازگشت کد را ساده‌تر و نزدیک به تعریف ریاضی می‌کند و برای مسائلی مانند پیمایش درخت‌ها مفید است. در مقابل، استفاده از بازگشت مصرف حافظه پشته را افزایش می‌دهد و احتمال سرریز پشته وجود دارد؛ همچنین معمولاً عملکرد نسخه تکراری بهتر است.